Categorical variables

Lecture notes

Foundations of Regression and Predictive Modeling

Table of contents

The basic problem . . . 1
Cheese sales and a single dummy variable . . . 2
More than two groups . . . 4
More than one categorical variable . . . 5
Adjusting for peak hours . . . 8
Adjustment as subtraction . . . 9
Interactions between two dummy variables . . . 9
Adding restaurant dummies . . . 11
Raw means versus baseline plus offsets . . . 14
Restaurant-specific priority effects . . . 14
Numerical and categorical variables together . . . 16
Toy grouped data . . . 16
Austin house prices . . . 20
Austin EDA . . . 23
Austin main-effects model . . . 24
Austin interaction model . . . 25
Correlation among predictors versus interaction . . . 28
What to carry forward . . . 29

The basic problem

A categorical variable names a condition, group, type, or status. Examples:

- which offer a customer saw on a cancellation page,
- whether a grocery item had an in-store display,
- whether a delivery order used priority dispatch,
- which restaurant produced the order,
- which Austin neighborhood a home is in.

Regression needs numbers. So a categorical variable has to be encoded before it can enter a model.

The main encoding in this module is **baseline plus offsets**.

Raw form lists the group values directly. For example, if Simone Biles is 56 inches tall, JJ Watt is 77 inches tall, and Yao Ming is 90 inches tall, raw form is just:

person	height
Simone Biles	56
JJ Watt	77
Yao Ming	90

Baseline/offset form chooses one value as the reference point. If Simone Biles is the baseline:

person	baseline/offset
Simone Biles	56
JJ Watt	56 + 21
Yao Ming	56 + 34

Same information. Different bookkeeping.

If JJ Watt is the baseline instead, the offsets change:

person	baseline/offset
Simone Biles	77 - 21
JJ Watt	77
Yao Ming	77 + 13

Changing the baseline changes the coefficients. It does not change the underlying group values.

Cheese sales and a single dummy variable

The first data example is 61 weeks of packaged cheese sales at a Dallas-area Kroger.

Question:

> How much more cheese sells when there is an in-store ad display?

Average weekly sales:

display?	mean sales
no	2341
yes	5577

In baseline/offset form:

group	baseline/offset
no display	2341
display	2341 + 3236

So display weeks sell about 3236 more units than no-display weeks.

To put this into a regression model, define a dummy variable:

$$\text{Display}_i = \begin{cases} 1, & \text{week } i \text{ had a display} \\ 0, & \text{otherwise.} \end{cases}$$

Now fit:

$$\text{Sales}_i = \beta_0 + \beta_1 \text{Display}_i + e_i.$$

The arithmetic is the whole point.

If there is no display, then Display = 0:

$$\widehat{\text{Sales}} = \beta_0 + \beta_1(0) = \beta_0.$$

If there is a display, then Display = 1:

$$\widehat{\text{Sales}} = \beta_0 + \beta_1(1) = \beta_0 + \beta_1.$$

For the cheese data:

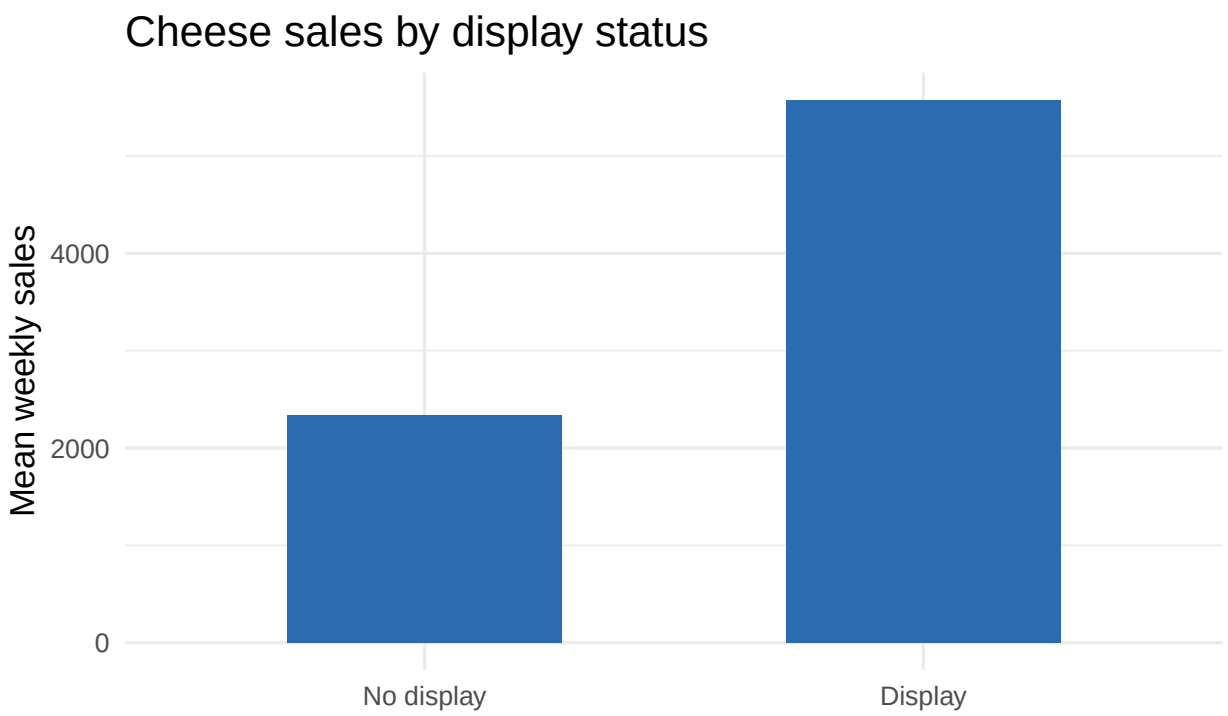
$$\widehat{\text{Sales}} = 2341 + 3236 \cdot \text{Display}.$$

So:

display?	fitted value
no	2341
yes	2341 + 3236 = 5577

The intercept is the baseline group mean. The display coefficient is the offset from the baseline.

In R, the dummy-variable mechanics look like this. Here we use the two lecture means directly, because the point is the encoding:



Usually we let R create the dummy variables from the factor:

```
model.matrix(~ disp, data = cheese_means)
```

```
  (Intercept) dispDisplay
1           1           0
2           1           1
attr(,"assign")
[1] 0 1
attr(,"contrasts")
attr(,"contrasts")$disp
[1] "contr.treatment"
```

R chooses a baseline level and creates the offset.

More than two groups

If a categorical variable has K levels, a regression model with an intercept uses:

- one baseline group,
- $K - 1$ dummy variables,
- $K - 1$ offsets from the baseline.

For example, suppose `Neighborhood` has three levels:

$$\text{Hyde Park}, \quad \text{Clarksville}, \quad \text{Barton Hills}.$$

If Hyde Park is the baseline, define two dummies:

$$C_i = \begin{cases} 1, & \text{Clarksville} \\ 0, & \text{otherwise} \end{cases}$$

and

$$B_i = \begin{cases} 1, & \text{Barton Hills} \\ 0, & \text{otherwise.} \end{cases}$$

The model is:

$$y_i = \beta_0 + \beta_1 C_i + \beta_2 B_i + e_i.$$

The fitted group means are:

group	C_i	B_i	fitted mean
Hyde Park	0	0	β_0
Clarksville	1	0	$\beta_0 + \beta_1$
Barton Hills	0	1	$\beta_0 + \beta_2$

We do not use three dummies for three groups, because the intercept already represents one group. The model needs a reference point.

The baseline is arbitrary. Changing it changes the intercept and offsets, but not the fitted group means.

More than one categorical variable

The delivery example uses several categorical variables at once.

Each row is one food-delivery order. The variables are:

- `Late`: minutes late,
- `Priority`: whether priority dispatch was used,
- `Peak`: whether the order was during peak hours,
- `Restaurant`: which restaurant made the order.

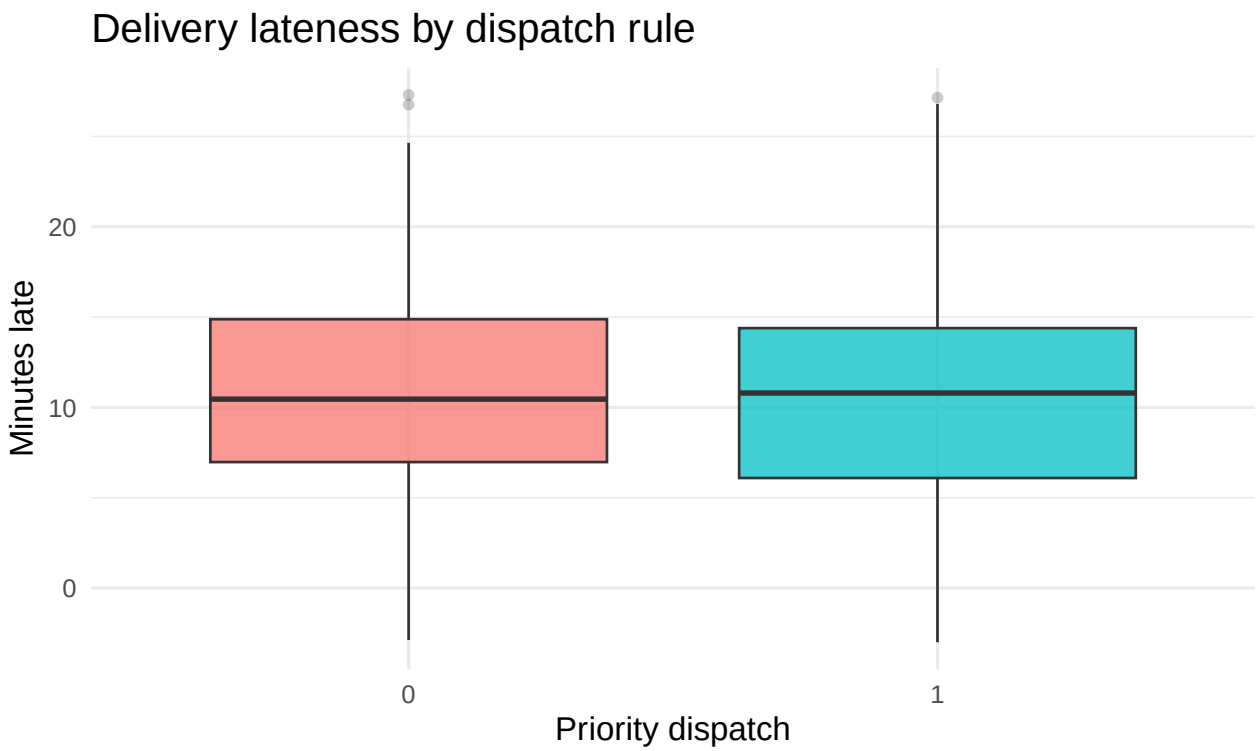
The business question is:

> How much does priority dispatch reduce late deliveries?

Read the data and inspect a few rows:

```
delivery <- read_csv(file.path(module_dir, "delivery_dispatch.csv")) |>
  mutate(
    Late = late_min,
    Restaurant = factor(Restaurant),
    Peak = factor(Peak, levels = c(0, 1)),
    Priority = factor(Priority, levels = c(0, 1))
  )
```

The naive comparison is just priority versus standard dispatch:



Start with the one-variable model:

$$\text{Late}_i = \beta_0 + \beta_1 \text{Priority}_i + e_i.$$

For these data:

$$\widehat{\text{Late}} = 11.2 - 0.9 \text{Priority}.$$

The model says priority dispatch is associated with only about 0.9 fewer minutes late.

That is the **overall** comparison: all priority orders versus all standard orders.

```
priority_only <- lm(Late ~ Priority, data = delivery)

tidy(priority_only)
```

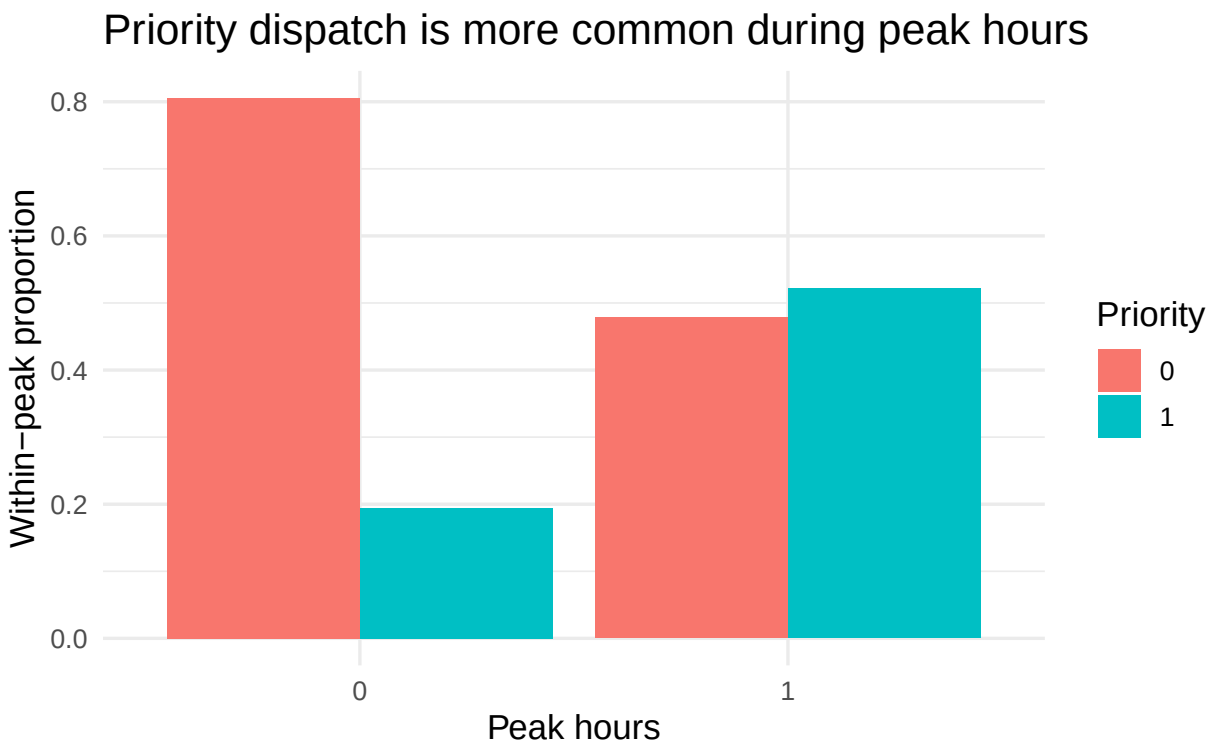
```
# A tibble: 2 x 5
  term        estimate std.error statistic p.value
  <chr>          <dbl>     <dbl>     <dbl>   <dbl>
1 (Intercept)   11.0       0.162     67.8  0
2 Priority1     -0.908     0.278     -3.26 0.00112
```

But priority dispatch is used more often during peak hours. Peak-hour orders are also later. So the naive priority comparison mixes together two differences:

- priority versus standard dispatch,
- peak-hour versus off-peak orders.

That is confounding. A confounder is related to the predictor we care about and to the outcome.

```
# A tibble: 4 x 4
# Groups:   Peak [2]
  Peak  Priority     n proportion
  <fct> <fct>    <int>      <dbl>
1 0     0          817      0.806
2 0     1          197      0.194
3 1     0          376      0.478
4 1     1          410      0.522
```



Adjusting for peak hours

Add `Peak` to the model:

$$\text{Late}_i = \beta_0 + \beta_1 \text{Priority}_i + \beta_2 \text{Peak}_i + e_i.$$

For these data:

$$\widehat{\text{Late}} = 8.9 - 3.8\text{Priority} + 7.6\text{Peak}.$$

Now the priority coefficient compares priority and standard dispatch **holding peak status fixed**.

The arithmetic shows the comparison:

Priority	Peak	fitted lateness
0	0	8.9
1	0	$8.9 - 3.8 = 5.1$
0	1	$8.9 + 7.6 = 16.5$
1	1	$8.9 - 3.8 + 7.6 = 12.7$

Within off-peak orders, priority changes the fitted value from 8.9 to 5.1, a difference of -3.8 minutes.

Within peak-hour orders, priority changes the fitted value from 16.5 to 12.7, also a difference of -3.8 minutes.

That is what a main-effects model assumes: the priority gap is the same in both peak statuses.

```
peak_adjusted <- lm(Late ~ Priority + Peak, data = delivery)

tidy(peak_adjusted)
```

```
# A tibble: 3 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)     8.65     0.146      59.3 0
2 Priority1      -3.55     0.234     -15.2 6.07e- 49
3 Peak1           7.34     0.223      32.9 8.54e-186
```

Adjustment as subtraction

Start with the adjusted model:

$$\widehat{\text{Late}} = 8.9 - 3.8\text{Priority} + 7.6\text{Peak}.$$

Move the peak-hour piece to the left:

$$\widehat{\text{Late}} - 7.6\text{Peak} = 8.9 - 3.8\text{Priority}.$$

The left side is fitted lateness after subtracting the part associated with peak hours.

The right side contains the partial priority relationship.

Overall relationship:

> compare all priority orders with all standard orders.

Partial relationship:

> compare priority and standard orders with the same peak status.

Those are different comparisons, so they can give different coefficients.

Interactions between two dummy variables

The main-effects model assumes the priority gap is the same off peak and during peak hours. The boxplot in the slides suggests the priority gap is larger during peak hours.

To allow that, add an interaction:

$$\text{Late}_i = \beta_0 + \beta_1\text{Priority}_i + \beta_2\text{Peak}_i + \beta_{12}\text{Priority}_i\text{Peak}_i + e_i.$$

The product term is 1 only when both dummies are 1:

Priority	Peak	Priority × Peak
0	0	0
1	0	0
0	1	0
1	1	1

So the fitted means are:

Priority	Peak	fitted mean
0	0	β_0
1	0	$\beta_0 + \beta_1$

Priority	Peak	fitted mean
0	1	$\beta_0 + \beta_2$
1	1	$\beta_0 + \beta_1 + \beta_2 + \beta_{12}$

The priority effect off peak is:

$$(\beta_0 + \beta_1) - \beta_0 = \beta_1.$$

The priority effect during peak hours is:

$$(\beta_0 + \beta_1 + \beta_2 + \beta_{12}) - (\beta_0 + \beta_2) = \beta_1 + \beta_{12}.$$

The interaction coefficient is the extra priority effect during peak hours.

For these data:

$$\widehat{\text{Late}} = 8.7 - 2.6\text{Priority} + 8.3\text{Peak} - 2.1\text{Priority} \cdot \text{Peak}.$$

Now compute the four fitted means:

Priority	Peak	fitted lateness
0	0	8.7
1	0	$8.7 - 2.6 = 6.1$
0	1	$8.7 + 8.3 = 17.0$
1	1	$8.7 - 2.6 + 8.3 - 2.1 = 12.3$

So:

$$\text{priority effect off peak} = 6.1 - 8.7 = -2.6,$$

and

$$\text{priority effect at peak} = 12.3 - 17.0 = -4.7.$$

Same predictor, different context, different fitted effect. That is an interaction.

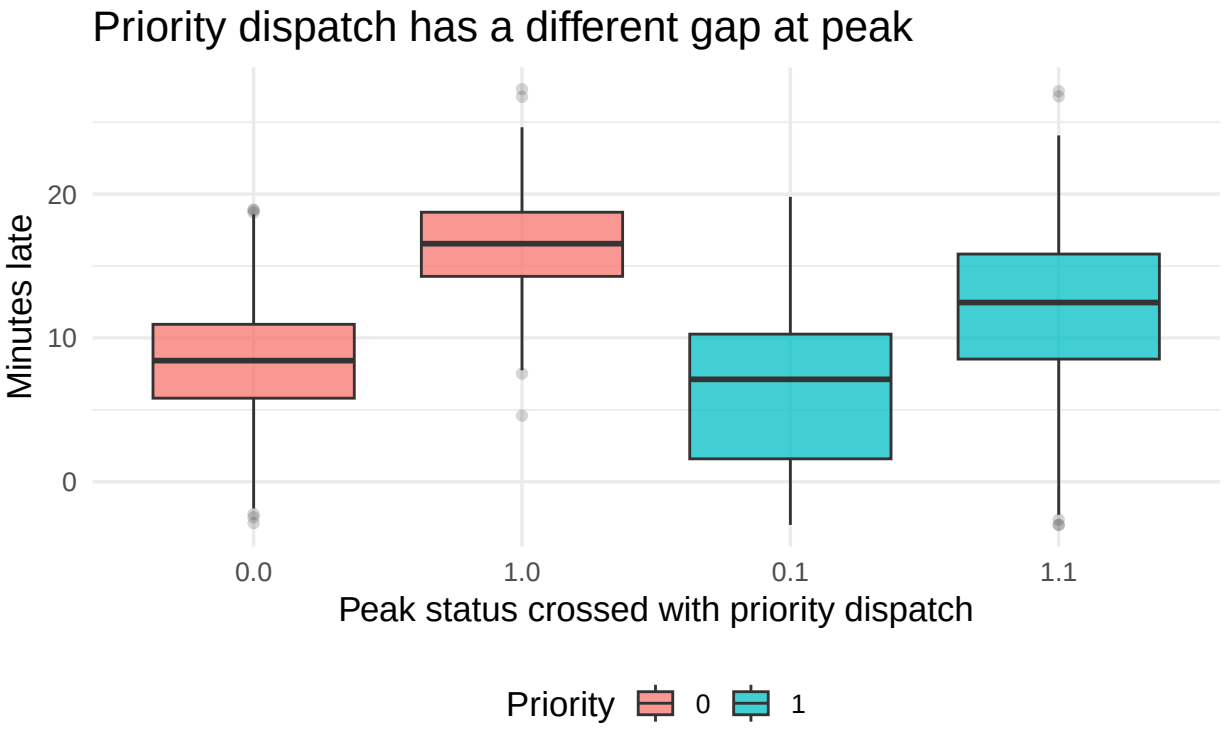
Fit the interaction model:

```
priority_peak <- lm(Late ~ Priority * Peak, data = delivery)

tidy(priority_peak)
```

```
# A tibble: 4 x 5
  term            estimate std.error statistic   p.value
  <chr>              <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)         8.41     0.153     54.8  0
2 Priority1          -2.29     0.348     -6.59 5.95e- 11
3 Peak1               8.11     0.273     29.7  6.83e-158
4 Priority1:Peak1    -2.28     0.468     -4.87 1.24e-  6
```

```
# A tibble: 4 x 3
  Priority Peak  fitted_late
  <fct>    <fct>       <dbl>
1 0        0            8.41
2 0        1           16.5
3 1        0            6.11
4 1        1           11.9
```



Adding restaurant dummies

Restaurants differ in baseline lateness. Some kitchens are slower, some pickups are easier, and some locations have worse traffic or parking.

Priority dispatch also differs by restaurant. That means a priority-versus-standard comparison is also mixing over restaurant mix.

Add restaurant dummies:

$$\text{Late} \sim \text{Priority} + \text{Peak} + \text{Priority} : \text{Peak} + \text{Restaurant}.$$

The fitted model from the slides has these main coefficients:

coefficient	estimate
intercept: R01, standard, off-peak	14.3
priority offset	-4.0
peak offset	7.5
priority:peak offset	-2.1

The restaurant offsets are relative to R01:

restaurant	offset	restaurant	offset
R02	-1.7	R08	-7.8
R03	-2.9	R09	-7.5
R04	-3.3	R10	-8.5
R05	-4.6	R11	-9.1
R06	-6.1	R12	-9.3
R07	-6.3		

The fitted value is assembled from pieces:

$$\begin{aligned}\widehat{\text{Late}} = {} & \text{baseline for R01, standard, off-peak} \\ & + \text{priority offset, if priority} \\ & + \text{peak offset, if peak} \\ & + \text{priority:peak offset, if both} \\ & + \text{restaurant offset.}\end{aligned}$$

For example, R12 has a restaurant offset of -9.3.

For a standard off-peak R12 order:

$$\widehat{\text{Late}} = 14.3 - 9.3 = 5.0.$$

For a priority peak-hour R12 order:

$$\widehat{\text{Late}} = 14.3 - 4.0 + 7.5 - 2.1 - 9.3 = 6.4.$$

The restaurant coefficient is a baseline shift. It adjusts the restaurant's typical lateness. It is not a restaurant-specific priority effect.

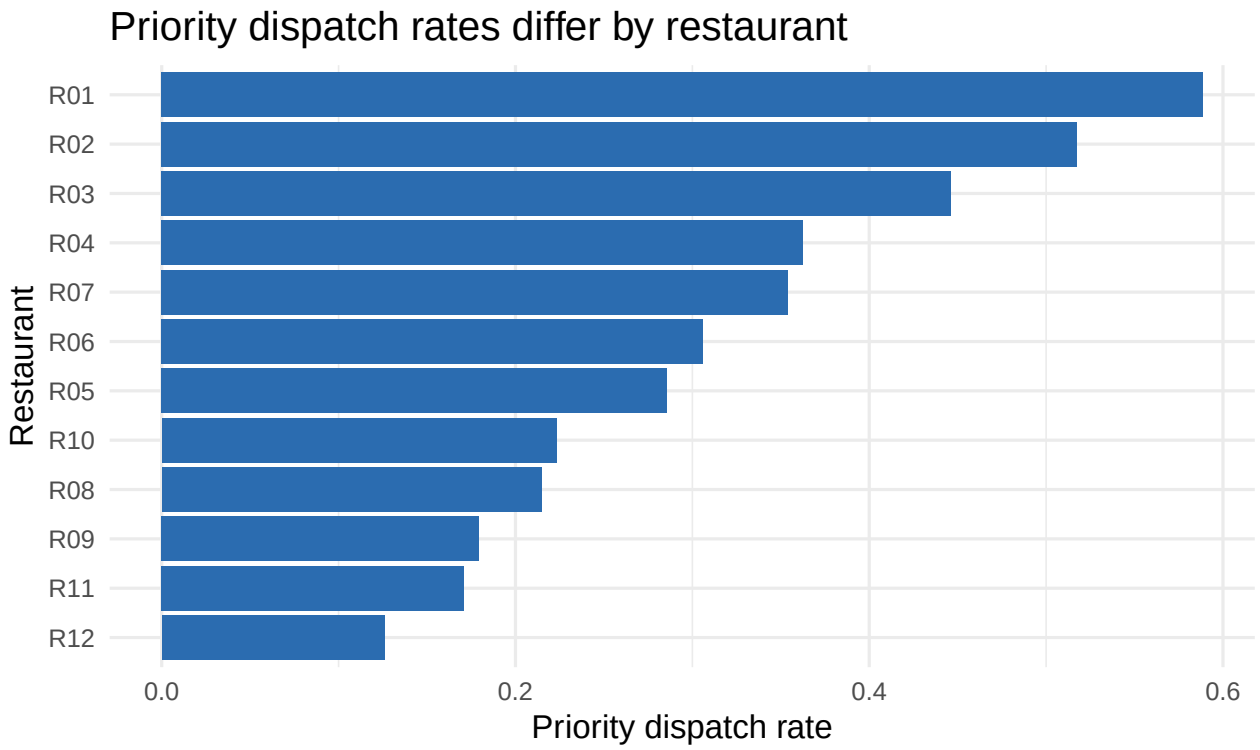
Fit the restaurant-adjusted model:

```r
restaurant_adjusted <- lm(Late ~ Priority * Peak + Restaurant, data = delivery)

tidy(restaurant_adjusted) |>
  select(term, estimate)
```

```
# A tibble: 15 x 2
   term            estimate
   <chr>              <dbl>
 1 (Intercept)        14.9
 2 Priority1          -3.98
 3 Peak1               6.85
 4 RestaurantR02      -1.66
 5 RestaurantR03      -2.81
 6 RestaurantR04      -4.09
 7 RestaurantR05      -5.63
 8 RestaurantR06      -6.05
 9 RestaurantR07      -7.12
10 RestaurantR08      -7.52
11 RestaurantR09      -8.37
12 RestaurantR10      -9.54
13 RestaurantR11      -9.75
14 RestaurantR12      -9.90
15 Priority1:Peak1    -2.08
```

Plotting restaurant-level priority rates is a good check for confounding:



Raw means versus baseline plus offsets

With priority status, peak status, and restaurant, raw means become awkward.

There are:

$$12 \text{ restaurants} \times 2 \text{ peak statuses} \times 2 \text{ dispatch rules} = 48$$

restaurant-by-peak-by-priority cells.

The raw-means path says: compute all 48 means, then compare pairs of means to reconstruct the priority effect.

That has three problems:

- some cells are small,
- the priority effect is scattered across many comparisons,
- adding another variable means slicing the data again.

It also implicitly allows a full three-way interaction:

$$\text{Priority} \times \text{Peak} \times \text{Restaurant}.$$

Baseline/offset form instead builds fitted means from components:

$$\widehat{\text{Late}} = \text{baseline} + \text{priority offset} + \text{peak offset} + \text{restaurant offset} + \text{chosen interaction offsets}.$$

Regression software does the assembly. The model output stays organized around the comparisons we care about.

Restaurant-specific priority effects

Now ask a new question:

> What if priority dispatch works better for some restaurants than others?

That requires a `Priority:Restaurant` interaction. The full working model in the slides is:

$$\text{Late} \sim \text{Priority} + \text{Peak} + \text{Restaurant} + \text{Priority} : \text{Peak} + \text{Priority} : \text{Restaurant}.$$

This includes two two-way interactions:

- `Priority:Peak`, so the priority effect can differ by peak status,
- `Priority:Restaurant`, so the priority effect can differ by restaurant.

```r
full_dispatch_model <- lm(
  Late ~ Priority + Peak + Restaurant + Priority:Peak + Priority:Restaurant,
  data = delivery
)

tidy(full_dispatch_model) |>
  select(term, estimate) |>
  filter(str_detect(term, "Priority"))
```

```
# A tibble: 13 x 2
   term                     estimate
   <chr>                       <dbl>
 1 Priority1                  0.0378
 2 Priority1:Peak1           -2.91
 3 Priority1:RestaurantR02   -1.82
 4 Priority1:RestaurantR03   -2.05
 5 Priority1:RestaurantR04   -3.02
 6 Priority1:RestaurantR05   -2.95
 7 Priority1:RestaurantR06   -4.11
 8 Priority1:RestaurantR07   -4.97
 9 Priority1:RestaurantR08   -6.55
10 Priority1:RestaurantR09   -6.26
11 Priority1:RestaurantR10   -6.96
12 Priority1:RestaurantR11   -7.99
13 Priority1:RestaurantR12   -7.11
```

The fitted priority coefficients include:

coefficient	estimate
priority effect at R01, off-peak	-0.7
extra priority effect at peak	-2.8
extra priority effect at R09	-6.4
extra priority effect at R10	-6.8
extra priority effect at R11	-6.7
extra priority effect at R12	-6.3

Here is the arithmetic for R10.

Off peak:

$$\text{priority effect} = -0.7 - 6.8 = -7.5.$$

At peak:

$$\text{priority effect} = -0.7 - 6.8 - 2.8 = -10.3.$$

The model-implied priority savings are standard dispatch minus priority dispatch:

restaurant	off-peak savings	peak savings
R10	7.5 min	10.3 min
R11	7.4 min	10.2 min
R09	7.1 min	9.9 min
R12	7.0 min	9.8 min
R07	5.9 min	8.7 min

These are fitted contrasts, holding restaurant and peak status fixed.

Numerical and categorical variables together

The next section combines one numerical predictor with one or more categorical predictors.

The grammar is:

- numerical variables create slopes,
- categorical variables create baseline shifts,
- interactions create context-specific slopes or context-specific offsets.

Toy grouped data

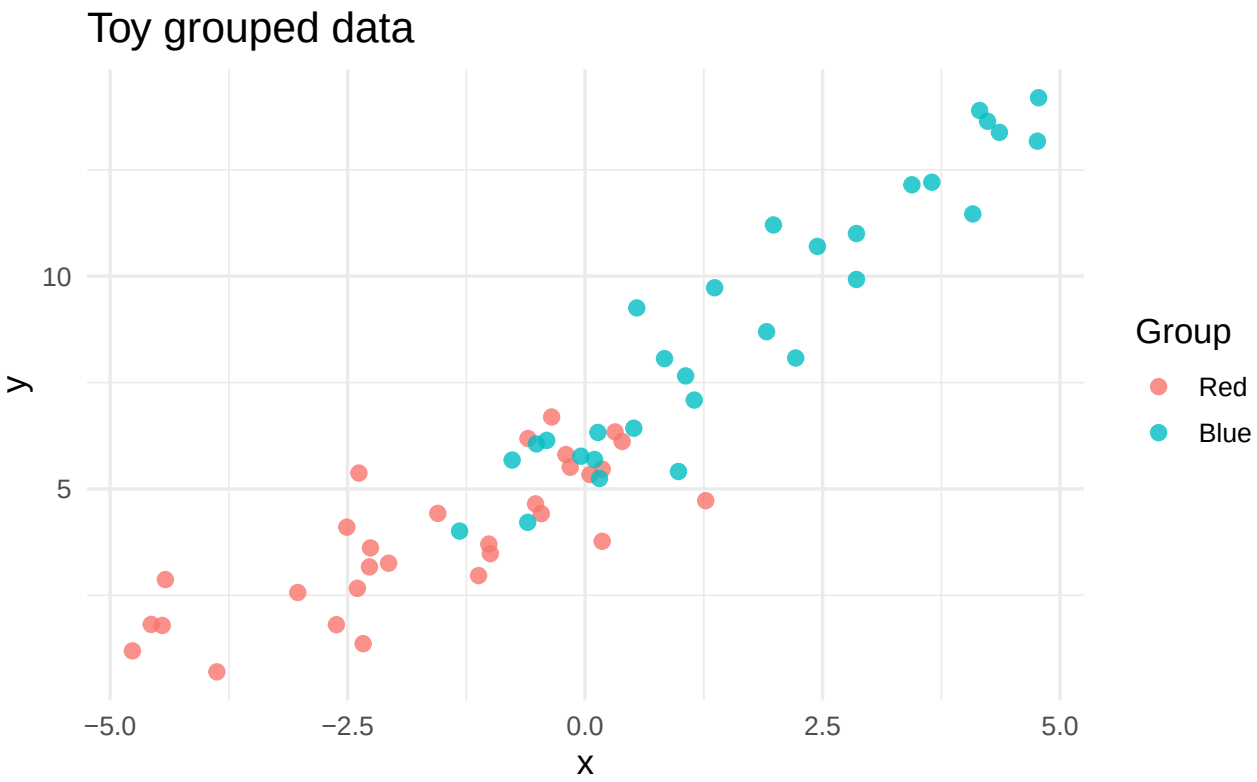
The toy data have:

- response `y`,
- numerical predictor `x`,
- group `Red` or `Blue`.

The blue group is shifted up and to the right, and the blue trend is visually steeper than the red trend.

Read and plot the toy data:

```
toy <- read_csv(file.path(module_dir, "toy_grouped_lines.csv")) |>
  mutate(Group = factor(Group, levels = c("Red", "Blue")))
```



First fit one line:

$$y_i = \beta_0 + \beta_1 x_i + e_i.$$

This treats red and blue points as one cloud. It ignores group structure.

```
toy_single <- lm(y ~ x, data = toy)

tidy(toy_single)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)     6.32    0.167       37.9 1.26e-42
2 x               1.38    0.0694      19.9 1.51e-27
```

Now add a dummy:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 \text{Blue}_i + e_i.$$

For red points, Blue $= 0$:

$$\hat{y} = \beta_0 + \beta_1 x.$$

For blue points, Blue $= 1$:

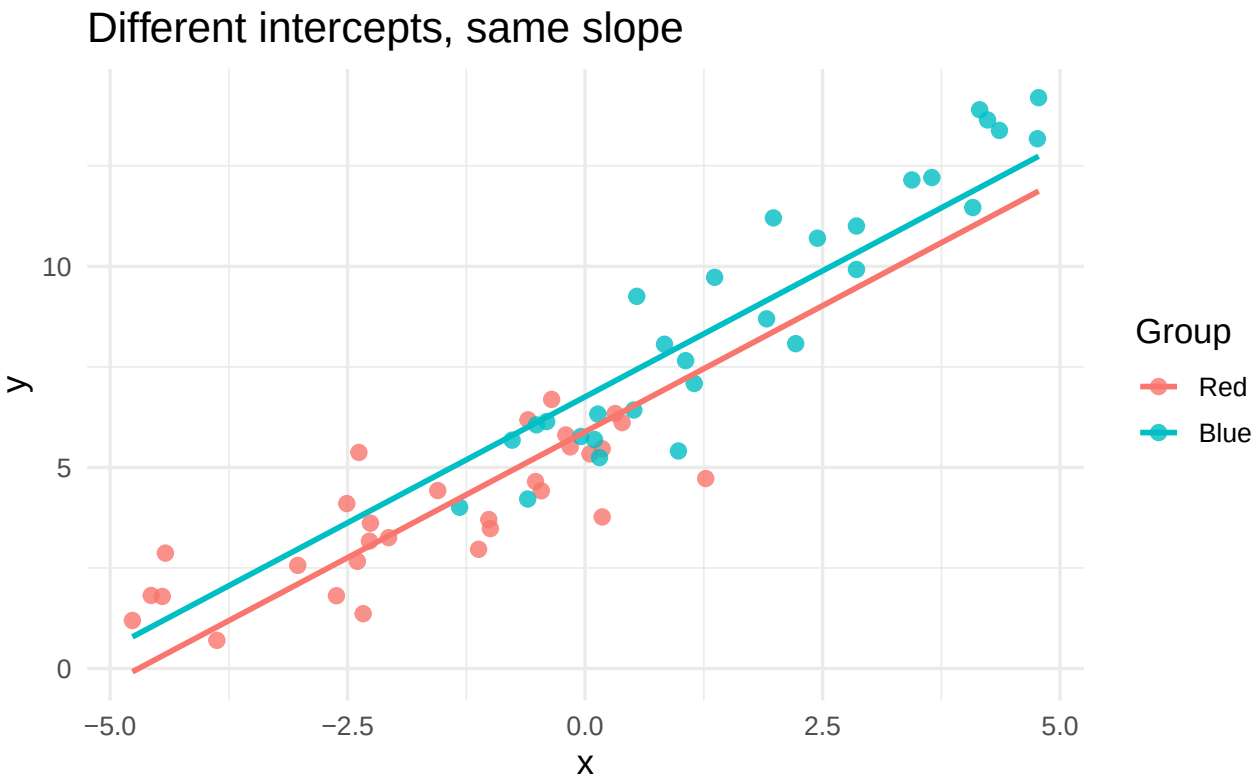
$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 = (\beta_0 + \beta_2) + \beta_1 x.$$

So the blue line has a different intercept, but the same slope. This is a main-effects model.

```
toy_main <- lm(y ~ x + Group, data = toy)

tidy(toy_main)
```

```
# A tibble: 3 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)    5.89     0.276      21.4  7.00e-29
2 x              1.25     0.0937     13.4  3.15e-19
3 GroupBlue      0.867    0.450       1.93 5.89e- 2
```



The plot suggests that blue rises faster with x. To allow that, add an interaction:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 \text{Blue}_i + \beta_3 x_i \text{Blue}_i + e_i.$$

Now the arithmetic changes.

For red points:

$$\hat{y} = \beta_0 + \beta_1 x.$$

For blue points:

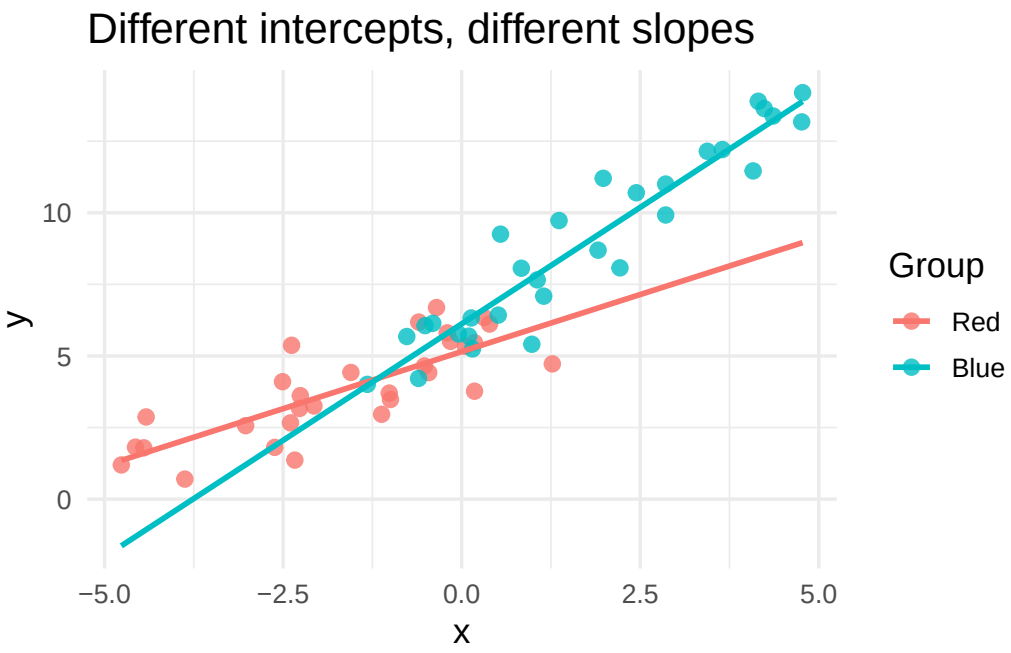
$$\begin{aligned}\hat{y} &= \beta_0 + \beta_1 x + \beta_2 + \beta_3 x \\ &= (\beta_0 + \beta_2) + (\beta_1 + \beta_3) x.\end{aligned}$$

The dummy coefficient is an intercept offset. The interaction coefficient is a slope offset.

```
toy_interaction <- lm(y ~ x * Group, data = toy)

tidy(toy_interaction)
```

```
# A tibble: 4 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)    5.15      0.264     19.5  1.22e-26
2 x              0.797     0.114      6.98 3.79e- 9
3 GroupBlue      0.970     0.369      2.63 1.11e- 2
4 x:GroupBlue    0.830     0.154      5.38 1.51e- 6
```




```
# A tibble: 2 x 3
  group intercept slope
  <chr>     <dbl> <dbl>
1 Red        5.15 0.797
2 Blue       6.12 1.63
```

In the fitted toy interaction model:

group	fitted line
Red	$\hat{y} \approx 5.15 + 0.80x$
Blue	$\hat{y} \approx 6.12 + 1.63x$

That is the same equation doing two jobs: one slope and intercept for red, another slope and intercept for blue.

Austin house prices

The Austin example uses the same grammar with real variables.

The data are a sample of Austin home sales collected by students for a class project last year. The sample focuses on three neighborhoods:

- Hyde Park,
- Clarksville,
- Barton Hills.

The variables in this section are:

- `PriceK`: sale price in thousands of dollars,
- `Sqft`: square footage,
- `MajorReno10`: whether the home had a major renovation in the last 10 years,
- `Neighborhood`: Austin neighborhood.

```
houses <- read_csv(file.path(module_dir, "austin_house_sales.csv")) |>
  mutate(
    Neighborhood = factor(Neighborhood, levels = c("Hyde Park", "Clarksville", "Barton Hills"))
    MajorReno10 = factor(MajorReno10, levels = c("No", "Yes")),
    Sqft100 = (Sqft - 1800) / 100,
    PricePerSqft = Price / Sqft
  )
```

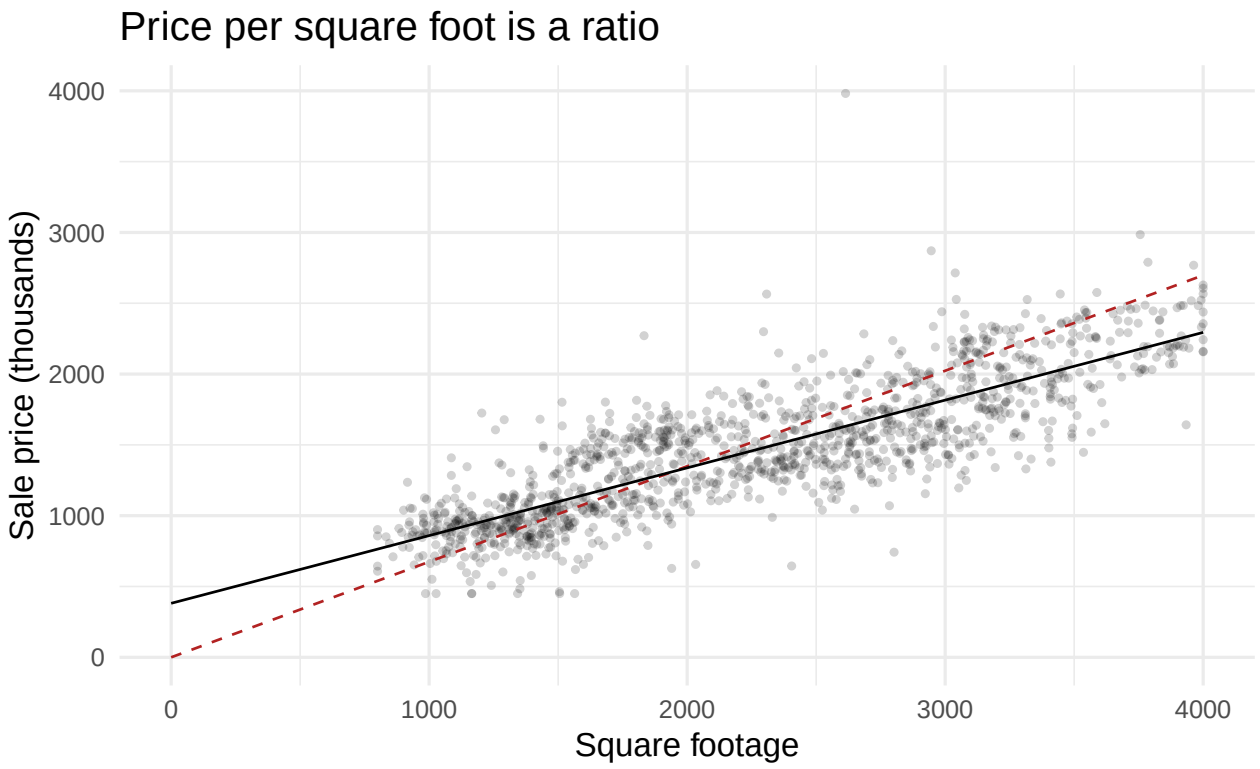
Realtors often quote a price-per-square-foot number:

$$\frac{\text{price}}{\text{square footage}}.$$

That sounds like a slope: dollars per extra square foot. But it is a ratio, not a marginal slope. It implicitly draws a line through zero:

$$\text{Price} = (\text{price per sqft}) \cdot \text{Sqft}.$$

A regression line does not have to pass through zero. It estimates both an intercept and a slope. That distinction matters when we ask what an additional square foot is associated with, holding the model structure fixed.



Square footage is centered:

$$\text{Sqft100} = \frac{\text{Sqft} - 1800}{100}.$$

So:

- `Sqft100 = 0` means 1800 square feet,
- `Sqft100 = 1` means 1900 square feet,
- `Sqft100 = -1` means 1700 square feet.

The slope is still measured per 100 square feet.

Centering changes the meaning of the intercept.

Without centering, a simple model like

$$\widehat{\text{PriceK}} = b_0 + b_1\text{Sqft}$$

has an intercept at Sqft $= 0$. Geometrically, the intercept is where the fitted line crosses the vertical axis. Substantively, that means the fitted price of a zero-square-foot home. That is an extrapolation to a house that does not exist.

With centering, a model like

$$\widehat{\text{PriceK}} = a_0 + a_1 \text{Sqft100}$$

has its intercept at Sqft100 = 0, which means Sqft = 1800. So the intercept is the fitted price for a typical-size home.

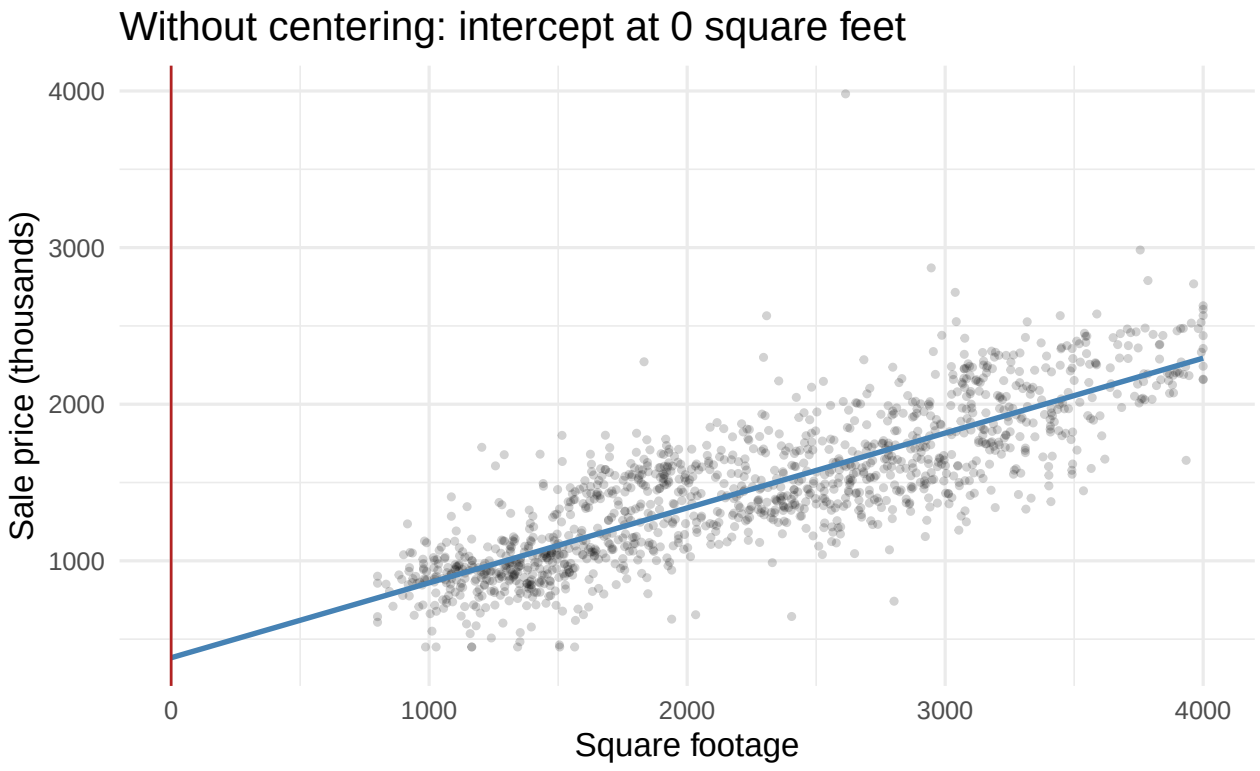
This matters even more with interactions, because dummy coefficients in interaction models refer to a specific value of the numerical predictor. Centering tells us which value.

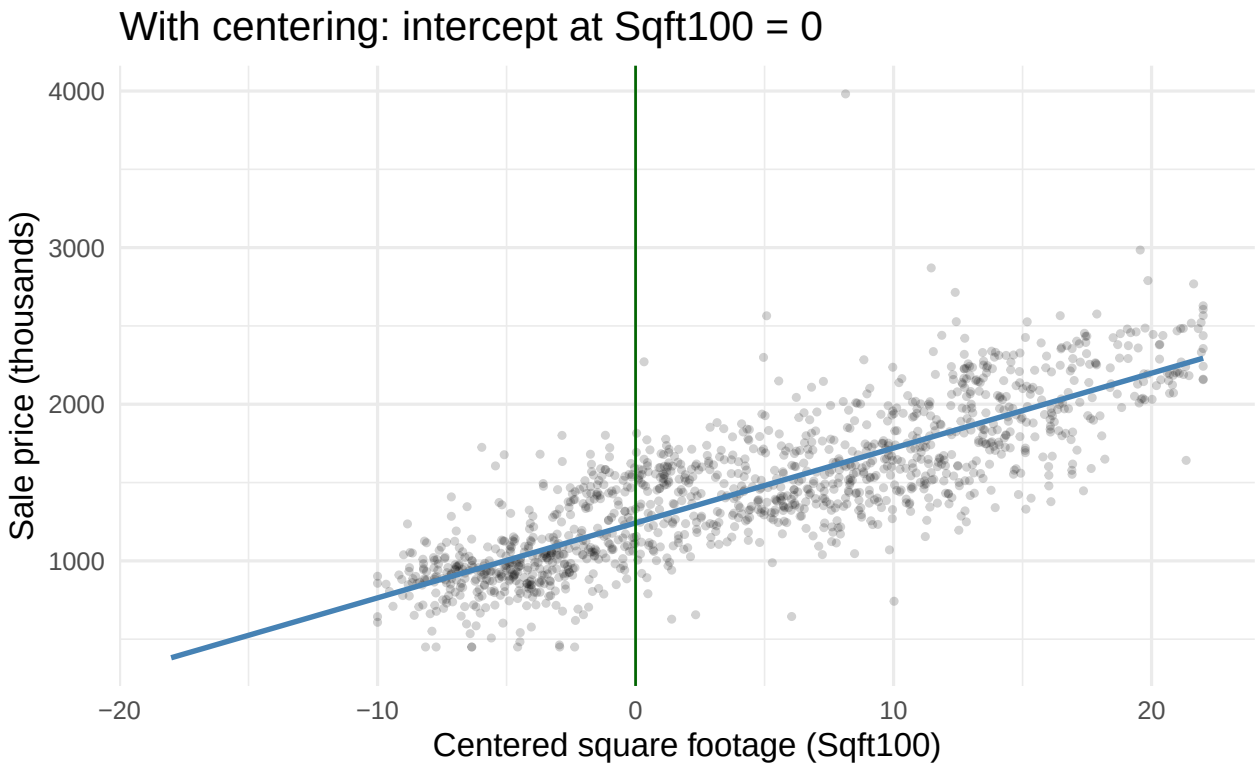
```
coef(simple_sqft_model)
```

```
(Intercept)        Sqft
381.2694862   0.4782357
```

```
coef(simple_centered_model)
```

```
(Intercept)     Sqft100
 1242.09384    47.82357
```





Austin EDA

Size and price differ by neighborhood:

neighborhood	mean sqft	mean price	mean price/sqft
Hyde Park	2065	1194K	\$604
Clarksville	2000	1289K	\$689
Barton Hills	2676	1870K	\$727

Barton Hills has larger houses and higher prices. Clarksville has similar average size to Hyde Park, but higher average price. Neighborhood cannot be ignored.

Size and price also differ by renovation status:

major renovation	mean sqft	mean price	mean price/sqft
No	1979	1297K	\$690
Yes	2692	1717K	\$650

Recently renovated homes are larger on average and more expensive on average. So renovation status is entangled with size.

Austin main-effects model

Start with:

$$\text{PriceK} \sim \text{Sqft100} + \text{MajorReno10} + \text{Neighborhood}.$$

For these data:

$$\widehat{\text{PriceK}} = 1050 + 35.2\text{Sqft100} + 161.8\text{MajorReno10} + \text{neighborhood offsets}.$$

This model allows different intercepts. But it assumes the same square-footage slope for renovated and unrenovated homes.

Using Hyde Park as the baseline neighborhood:

situation	fitted line
Hyde Park, no major renovation	$1050 + 35.2\text{Sqft100}$
Hyde Park, major renovation	$(1050 + 161.8) + 35.2\text{Sqft100}$

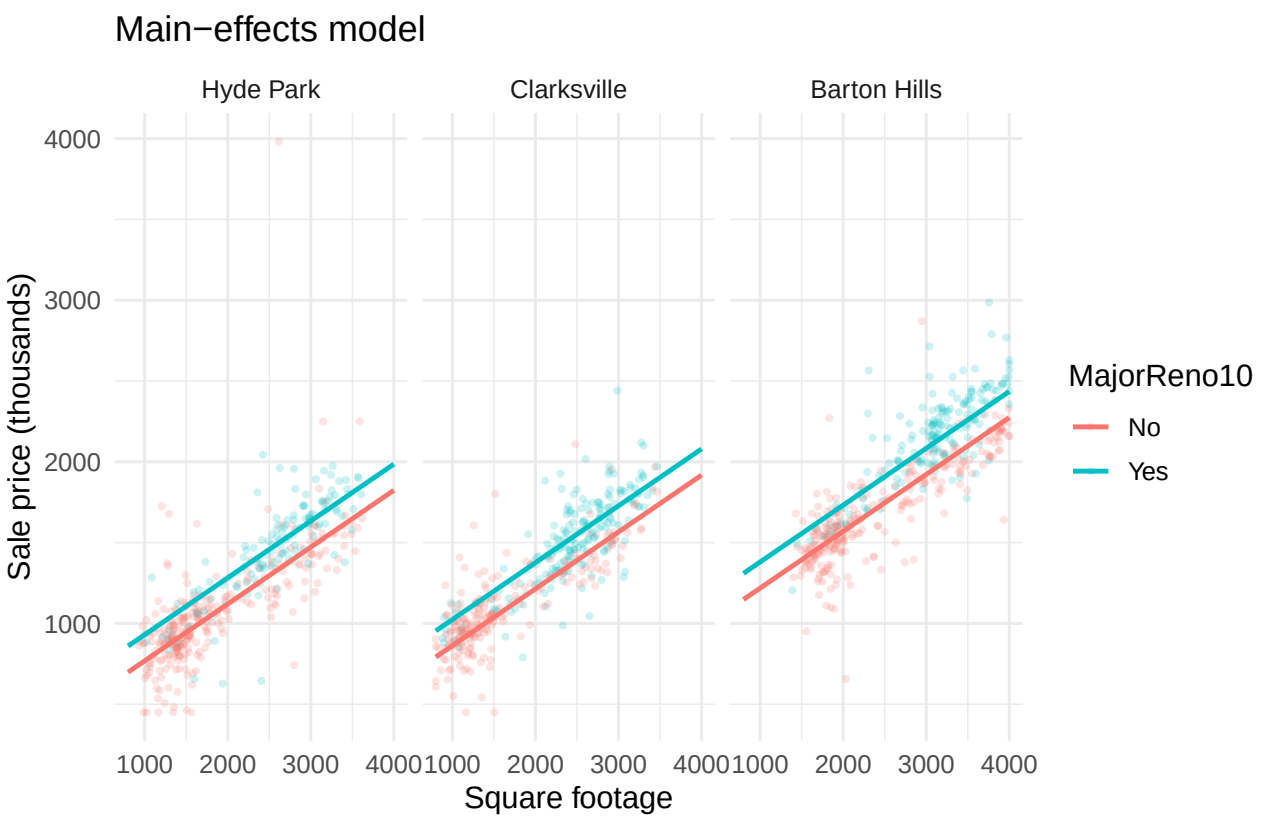
The renovation coefficient shifts the intercept by about 161.8 thousand dollars. It does not change the slope.

So under this model, every extra 100 square feet is associated with about $35.2K more, whether or not the home had a recent major renovation.

```
main_model <- lm(PriceK ~ Sqft100 + MajorReno10 + Neighborhood, data = houses)

tidy(main_model)
```

```
# A tibble: 5 x 5
  term                     estimate std.error statistic   p.value
  <chr>                       <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)                1050.      9.81     107.   0
2 Sqft100                      35.2     0.779     45.1  4.00e-268
3 MajorReno10Yes              162.     12.3       13.2  2.18e- 37
4 NeighborhoodClarksville      93.9    13.2        7.13 1.63e- 12
5 NeighborhoodBarton Hills    450.     13.5       33.2  1.15e-175
```



Austin interaction model

The main-effects fitted lines are parallel within each neighborhood. The plot suggests that may be too restrictive: the square-footage premium appears steeper for recently renovated homes.

Let the square-footage slope differ by renovation status:

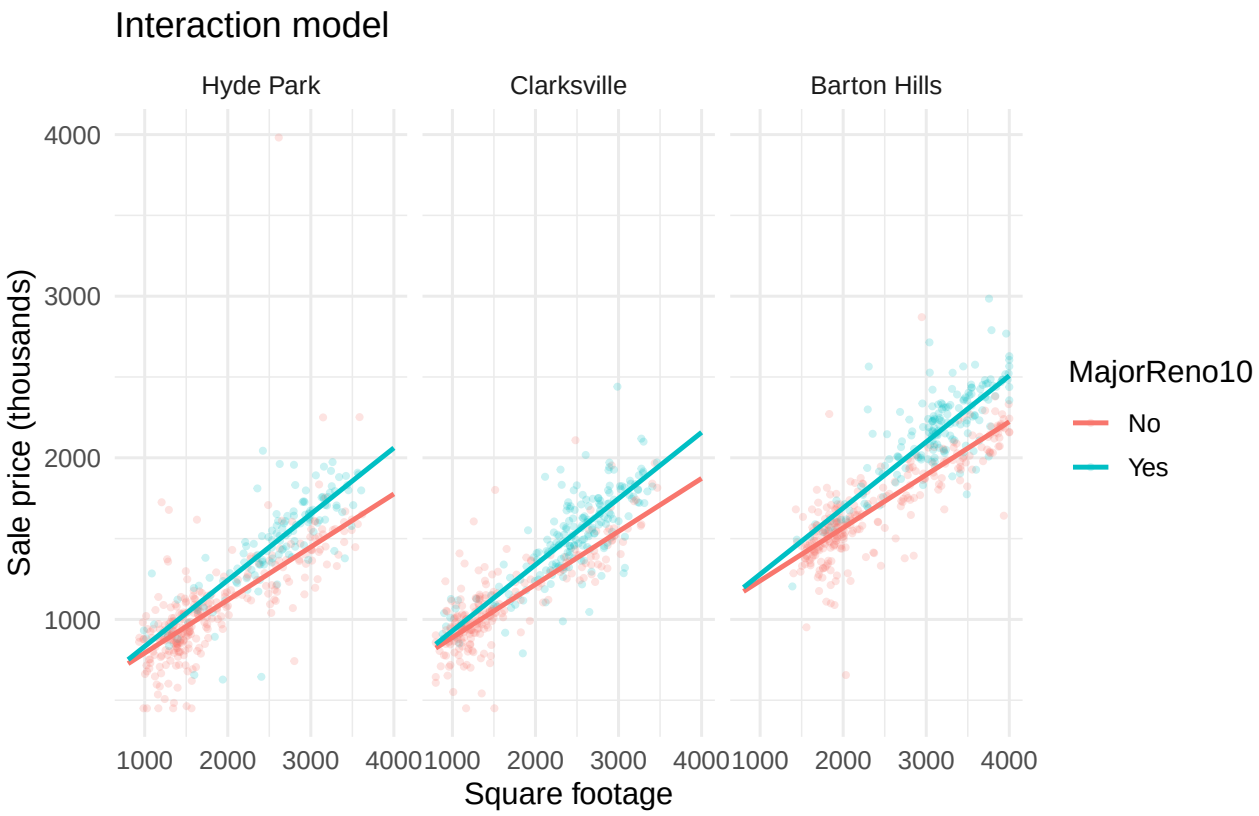
$$\text{PriceK} \sim \text{Sqft100} * \text{MajorReno10} + \text{Neighborhood}.$$

For these data, let $R = \text{MajorReno10}$, $C = \text{Clarksville}$, and $B = \text{Barton Hills}$.

```
interaction_model <- lm(PriceK ~ Sqft100 * MajorReno10 + Neighborhood, data = houses)

tidy(interaction_model)
```

```
# A tibble: 6 x 5
  term                      estimate std.error statistic   p.value
  <chr>                        <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)                 1055.       9.74     108.   0
2 Sqft100                       32.8      0.889     36.9  2.00e-204
3 MajorReno10Yes               106.      16.0        6.61 5.51e- 11
4 NeighborhoodClarksville       96.1     13.0        7.38 2.85e- 13
5 NeighborhoodBarton Hills     447.      13.4       33.3  2.29e-176
6 Sqft100:MajorReno10Yes         8.13     1.52       5.36 1.01e-  7
```



$$\widehat{\text{PriceK}} = 1054.6 + 32.8\text{Sqft100} + 105.9R + 96.1C + 446.7B + 8.1(\text{Sqft100} \cdot R).$$

For Hyde Park, $C = 0$ and $B = 0$.

No major renovation means $R = 0$:

$$\widehat{\text{PriceK}} = 1054.6 + 32.8\text{Sqft100}.$$

Major renovation means $R = 1$:

$$\begin{aligned}\widehat{\text{PriceK}} &= 1054.6 + 32.8\text{Sqft100} + 105.9 + 8.1\text{Sqft100} \\ &= (1054.6 + 105.9) + (32.8 + 8.1)\text{Sqft100}.\end{aligned}$$

So in Hyde Park:

renovation status	intercept	slope per 100 sqft
No	1054.6	32.8
Yes	1160.5	40.9

For Barton Hills, $C = 0$ and $B = 1$, so the $446.7B$ term enters the intercept.

No major renovation means $R = 0$:

$$\widehat{\text{PriceK}} = (1054.6 + 446.7) + 32.8\text{Sqft100}.$$

Major renovation means $R = 1$:

$$\begin{aligned}\widehat{\text{PriceK}} &= 1054.6 + 446.7 + 105.9 + (32.8 + 8.1)\text{Sqft100} \\ &= (1054.6 + 446.7 + 105.9) + 40.9\text{Sqft100}.\end{aligned}$$

So in Barton Hills:

renovation status	intercept	slope per 100 sqft
No	1501.3	32.8
Yes	1607.2	40.9

Neighborhood changes the intercept. Renovation changes the intercept and the slope.

The same table can be assembled directly from the fitted coefficients:

```
# A tibble: 6 x 4
  Neighborhood MajorReno10 intercept slope
  <chr>        <chr>           <dbl> <dbl>
1 Hyde Park    No              1055.  32.8
2 Hyde Park    Yes             1160.  40.9
3 Clarksville  No              1151.  32.8
4 Clarksville  Yes             1257.  40.9
5 Barton Hills No              1501.  32.8
6 Barton Hills Yes             1607.  40.9
```

Now be careful with the coefficient on `MajorReno10`. The coefficient is 105.9. A tempting reading is:

> recent major renovation is worth \$105.9K.

But with an interaction, that is not quite right. The renovation gap is not read off from one coefficient by itself. It comes from comparing two fitted prices for houses of the same size, in the same neighborhood.

Fix a neighborhood and a value of `Sqft100`. Then the fitted price for a renovated home is:

$$\widehat{\text{PriceK}}_{\text{reno}} = \text{same neighborhood offset} + 1054.6 + 32.8\text{Sqft100} + 105.9 + 8.1\text{Sqft100}.$$

The fitted price for a non-renovated home of the same size, in the same neighborhood, is:

$$\widehat{\text{PriceK}}_{\text{no reno}} = \text{same neighborhood offset} + 1054.6 + 32.8\text{Sqft100}.$$

Now subtract:

$$\begin{aligned}\text{renovation gap} &= \widehat{\text{PriceK}}_{\text{reno}} - \widehat{\text{PriceK}}_{\text{no reno}} \\ &= 105.9 + 8.1\text{Sqft100}.\end{aligned}$$

So 105.9 is the renovation gap only when `Sqft100 = 0`.

Because `Sqft100 = 0` means 1800 square feet, the `MajorReno10` coefficient of 105.9 means:

> among 1800-square-foot homes in the same neighborhood, recently renovated homes are fitted about \$105.9K higher than non-renovated homes.

At 1400 square feet, `Sqft100 = -4`, so:

$$105.9 + 8.1(-4) = 73.5.$$

The fitted renovation gap is about \$73.5K for a 1400-square-foot home.

At 2200 square feet, `Sqft100 = 4`, so:

$$105.9 + 8.1(4) = 138.3.$$

The fitted renovation gap is about \$138.3K for a 2200-square-foot home.

Correlation among predictors versus interaction

The tennis example separates two ideas students often confuse.

Correlation among predictors means the predictors move together. It creates causal confusion: it is hard to tell which predictor is responsible for the change in the outcome.

Interaction means the effect of one predictor depends on the level of another predictor. It is causal complexity: the answer really is context-specific.

The tennis setup has:

- grip: semi-western or western,
- stance: side-on/neutral or open,
- outcome: successful deep target hits out of 100 forehands.

In the bad experiment, the coach only gives two blocks:

stance	semi-western grip	western grip
side-on/neutral	58	–
open	–	72

You did better in the western/open block. But grip and stance are perfectly correlated. The first block always uses semi-western plus side-on/neutral. The second block always uses western plus open.

So the difference could be due to grip, stance, or both. The design cannot separate them.

A better experiment uses all four combinations:

stance	semi-western grip	western grip
side-on/neutral	66	58
open	55	75

Now each grip appears with each stance.

The stance effect for semi-western grip is:

$$55 - 66 = -11.$$

The stance effect for western grip is:

$$75 - 58 = 17.$$

Open stance hurts with semi-western grip and helps with western grip. That is an interaction.

The grip effect also depends on stance.

Within side-on/neutral stance:

$$58 - 66 = -8.$$

Within open stance:

$$75 - 55 = 20.$$

Same grip change, different effect. Again: interaction.

You can have correlated predictors with no interaction. You can have an interaction in a balanced design with no predictor correlation. You can also have both at once.

What to carry forward

Categorical variables enter regression through dummy variables.

The intercept is the baseline. Dummy coefficients are offsets from that baseline. With K groups, a model with an intercept uses $K - 1$ dummies.

With multiple categorical variables, fitted means are assembled from a baseline plus whichever offsets apply.

Interactions change what coefficients mean:

- with two dummies, an interaction creates context-specific group differences;

- with a numerical variable and a dummy, an interaction creates context-specific slopes.

Centering a numerical predictor does not change the fitted relationship. It changes the meaning of zero. That makes the intercept interpretable, and in interaction models it makes dummy coefficients interpretable too.